Benito André Pepe

2311720

1. A ideia é que é possível converter qualquer função, depois de tirar muitas abstrações (como loops e definições de funções), para o formato de funções parciais, que são equivalentes a uma máquina de Turing, que só usam operações básicas e algum tipo de rastreamento e controle de fluxo. A partir desse ponto, é possível percorrer cada instrução em sequência, enquanto houver instruções (como um while), e executar qualquer tipo de função.

2.

Partindo do teorema original: $f(x) = U(\mu y[T(e, x, y) = 0])$

U (função extratora) e T (predicado de Kleene) são funções primitivas recursivas e μ é a operação de minimização ilimitada, que pode gerar um loop.

Como os numerais de Church já são, por definição, funções que aplicam outras um número limitado de vezes, a recursão primitiva pode ser representada por eles. Assim, tanto U quanto T podem ser definidos usando eles.

Para μ, posso utilizar o combinador Ω, para fazer as iterações em quantidade indefinida na minimização ilimitada. Cada iteração pode ser representada pela função a seguir:

$\text{Step} = \lambda s.\, \lambda y.\, \text{If}\left(\text{IsZero}\ (T_\lambda\ \overline{e}\ x\ y)\right)\ y\ \left(s\ (\text{Succ}\ y)\right)$

$\text{Min} = \Omega\ \text{Step}$

Sendo que o trecho $T_\lambda\ \overline{e}\ x\ y$ representa a condição se o programa "e" com entrada "x" e histórico "y" para.

Assim, é possível juntar essas funções para formar a estrutura completa:

$F = \lambda x.\, U_\lambda\ \left(\Omega\ \text{Step}\ \overline{0}\right)$

$F = \lambda x.\, U_\lambda\ \left(\Omega\ \left(\lambda s.\, \lambda y.\, \text{If}\left(\text{IsZero}\ (T_\lambda\ \overline{e}\ x\ y)\right)\ y\ \left(s\ (\text{Succ}\ y)\right)\right)\ \overline{0}\right)$

Assim, com apenas uma recursão ilimitada, qualquer função poderia ser computada.

3. Primeiro, é necessário definir um caso base para uma lista com aridade 0, que também vai representar o fim da lista.

$L_0 = \text{nil}$

Para representar os índices seguintes da lista, é possível utilizar uma sublista como elemento da parificação, no estilo:

$L_n = [x, L_{n-1}]$

Assim, uma lista com aridade 1 seria algo como $L_1 = [0, \text{nil}]$, e para dois elementos, algo como $L_2 = [0, [1, \text{nil}]]$, e assim por diante.

4. Usando o operador de ponto fixo $Y = \lambda f.\,(\lambda x.\,f\,(x\,x))(\lambda x.\,f\,(x\,x))$:

$$f(n) = Y\,\left(\lambda f.\,\lambda n.\,\text{If}\,(\text{IsZero}\,n)\,\overline{0}\,\left(\text{Add}\,n\,\left(f\,(\text{Pred}\,n)\right)\right)\right)$$

f itera recursivamente sobre n, retornando 0 se n for zero, ou aplicando a soma com a iteração anterior.

$$\text{Fib} =$$

$$Y\,\left(\lambda f.\,\lambda n.\,\text{If}\,\left(\text{IsZero}\,(\text{Pred}\,n)\right)\,n\,\left(\text{Add}\,\left(f\,(\text{Pred}\,n)\right)\,\left(f\left(\text{Pred}\,(\text{Pred}\,n)\right)\right)\right)\right)$$

A função Fib se chama recursivamente duas vezes, através da variável f. Ela para de n = 0 ou 1, porque o lambda-calculus não aceita números negativos, então o predecessor deles é 0. E a cada iteração, se chama recursivamente em n e n-1 somando os dois valores.

$$\text{Comb} =$$

$$Y\left(\lambda c.\,\lambda n.\,\lambda k.\,\text{If}\,(\text{IsZero}\,k)\,\overline{1}\,\left(\text{If}\,(\text{IsZero}\,n)\,\overline{0}\,\left(\text{Add}\,\left(c\,(\text{Pred}\,n)\,(\text{Pred}\,k)\right)\,(c\,(\text{Pred}\,n)\,k)\right)\right)\right)$$

Comb é definida com base na identidade recursiva de Pascal. Em resumo, se k = 0, retorna 1, se n = 0 e k != 0, retorna 0, senão, se chama recursivamente e soma os resultados.

5. Usando o operador de ponto fixo $Y = \lambda f.\,(\lambda x.\,f\,(x\,x))(\lambda x.\,f\,(x\,x))$:

$$\text{AckStep} =$$

$$\lambda a.\,\lambda m.\,\lambda n.\,\text{If}\,(\text{IsZero}\,m)\,(\text{Succ}\,n)\,\left(\text{If}\,(\text{IsZero}\,n)\,\left(a\,(\text{Pred}\,m)\,\overline{1}\right)\,\left(a\,(\text{Pred}\,m)\,\left(a\,m\,(\text{Pred}\,n)\right)\right)\right)$$

$$\text{Ack} = Y\,\text{AckStep}$$

Sendo que m e n são numerais de Church e a é a própria função de Ackermann que é passada para fazer as chamadas recursivas. O combinador de ponto fixo Y é aplicado na função para formar a recursão, injetando os parâmetros.

A(0,n) = n+1 é definido como If (IsZero m) (Succ n)

A(m+1,0) = A(m,1) é definido como If (IsZero n) $\left(\text{a (Pred m) } \overline{1}\right)$

A(m+1,n+1) = A(m,A(m+1,n) é definido como $\left(\text{a (Pred m) } \left(\text{a m (Pred n)}\right)\right)$

6.

Exemplos:

Soma:

$+(x, 0) = x$

$+\big(x, \text{Suc}(y)\big) = \text{Suc}\big(+(x, y)\big)$

Multiplicação, que é a soma repetida, usando a função +:

$*\,(x, 0) = 0$

$*\,\big(x, \text{Suc}(y)\big) = +(*\,(x, y), x)$

Exponenciação, que é a multiplicação repetida, usando a função *.

$\text{Exp}(x, 0) = \text{Suc}(0)$

$\text{Exp}\big(x, \text{Suc}(y)\big) =*\,(\text{Exp}(x, y), x)$

Predecessor:

$\text{Pred}(0) = 0$

$\text{Pred}\big(\text{Suc}(x)\big) = x$

Monus:

$\text{Monus}(x, 0) = x$

$\text{Monus}\big(x, \text{Suc}(y)\big) = \text{Pred}\big(\text{Monus}(x, y)\big)$

Para formar funções primitivas recursivas, é necessário tem a função zero, sucessor, projeção, e propriedades de composição, que o enunciado diz que o modelo possui, e recursão primitiva, que também é disposta pelo sistema de equações. De forma geral, é possível podemos criar uma forma da função para o caso base definindo o segundo parâmetro como 0 e o caso de passo indutivo com a função Suc.

Para que a forma seja capaz de definir funções parcialmente recursivas, ela precisa ser capaz de aplicar as regras uma quantidade de vezes indeterminada, em outras palavras, ela precisa ter um equivalente ao operador de minimização limitada. No processo de definição das funções desse modelo, não existe um momento em que é definido um limite de quantidade de execuções, como nas funções parcialmente recursivas. As funções são executadas até chegar em um valor numérico fechado que não chame outras funções. Se uma das entradas da função não for tratada para eventualmente parar, o programa entra em loop e não para, então o modelo é parcialmente recursivo.

7. O teorema da recursão é o que justifica que a função é parcialmente recursiva. Sem ele, não seria possível definir isso. Ele é o que garante que para qualquer função, além da equação funcional, existe uma validação semântica. O Teorema da Recursão de Kleene diz que para qualquer operador computável funcional, sempre existirá uma menor função parcial, ou ponto fixo, para satisfazer a equação $F = E(F)$. Sem identificar a possibilidade de uma recursão indefinida, não seria possível definir se o modelo alcança as restrições das funções parcialmente recursivas.

10. Seja F o conjunto de funções que podem ser representadas com Funções Parcialmente Recursivas, C as funções representadas com λ-calculus e T as funções representadas pela máquina de Turing. Conforme provado, tanto os modelos do λ-calculus quanto da máquina de Turing são equivalentes a Funções Parcialmente Recursivas. Portanto, todas as funções contidas em T fazem parte do conjunto F e todas as funções de C fazem parte do conjunto T, e vice-versa. Pela propriedade transitiva da igualdade, T equivale a F e F equivalente a C, então C equivale a T. Os modelos são equivalentes.

11.

1) F. O ponto fixo normalmente entra em loop em várias funções, mas nem todas. Se N for uma função que ignora os argumentos, pela propriedade de lazy evaluation, é possível fazer uma redução sem precisar avaliar a parte da expressão que entra em loop. Nesses casos, é possível chegar a uma forma normal.

2) V. Por definição, se esses termos ainda pudessem ser reduzidos, eles não poderiam ser representações de valores, nem seriam usados como base para outras operações.

3) F. Se a expressão $((\lambda x.\, z)((\lambda x.\, x\, x)(\lambda x.\, x\, x)))$ for reduzida pela esquerda, ele pode ser reduzida normalmente para z, já que x não é utilizado na expressão, mas se for reduzida pela direita, entraria em loop e não seria possível obter a forma normal.

4) F.

$0 = \lambda a.\, \lambda b.\, b.$

$(\mathrm{Y\ Suc})0 \rightarrow_\beta \lambda y.\, 0\big((\mathrm{Y\ Suc})0y\big)$

$\rightarrow_\beta \lambda y.\, (\lambda a.\, \lambda b.\, b)\big((\mathrm{Y\ Suc})0y\big)$

$\rightarrow_\beta \lambda y.\, \lambda b.\, b$

12.

Expressão: $K = (\lambda x.\, \lambda y.\, x\ x)(\lambda x.\, \lambda y.\, x\ x)$

Demonstração:

$\big((\lambda x.\, \lambda y.\, x\ x)(\lambda x.\, \lambda y.\, x\ x)\big) M$

$\rightarrow_\beta \big(\lambda y.\, (\lambda x.\, \lambda y.\, x\ x)(\lambda x.\, \lambda y.\, x\ x)\big) M$

(Como y não existe no corpo da função, M é descartado)

$\rightarrow_\beta (\lambda x.\, \lambda y.\, x\ x)(\lambda x.\, \lambda y.\, x\ x) \rightarrow_\beta K$

Esse programa pode ser visto como a impressão dele mesmo porque, dada a execução, o "resultado final" é o próprio programa K.

13. O array pode ser interpretado matematicamente como uma função que associa índices a valores. Escrever no array é semelhante a criar um novo array (uma nova função) que também inclui o novo valor. A função New é necessária para definir quando o array "acaba". A função Get só precisa executar a função do array em um índice específico.

$\text{Insert} = \lambda A.\, \lambda k.\, \lambda v.\, \lambda i.\, (==\ i\ k)\ v\ (A\ i)$

$\text{New} = \lambda i.\, \bot$

$\text{Get} = \lambda A.\, \lambda i.\, A\ i$

14.

$\text{Insert2D} = \lambda A.\, \lambda x.\, \lambda y.\, \lambda v.\, \lambda i.\, \lambda j.\, (==\ i\ x)\ \big((==\ j\ y)\ v\ (A\ i\ j)\big)\ (A\ i\ j)$

$\text{Get2D} = \lambda A.\, \lambda x.\, \lambda y.\, A\ x\ y$

$\text{New2D} = \lambda x.\, \lambda y.\, \bot$

A principal diferença é que, para adicionar novas dimensões, é preciso adicionar novas verificações com ==, e novos elementos do tipo λz que funcionam como o $\lambda x.\, \lambda y$. Em uma perspectiva de elegância e simplicidade, se eu definir sucinto como o tamanho final das funções em quantidade de operadores e operandos, considero que adicionar mais dimensões ao array não é uma tarefa complexa, mas é trabalhosa e não é escalável. A expressão final são variações da mesma coisa, mas fica longa, repetitiva e confusa, então não considero sucinta.

15.

$$D = \lambda x.\,(xx) = S(\lambda x.\,x)(\lambda x.\,x)$$

$$D = SII$$

Para definir I, usando S e K:

K: $\alpha \rightarrow (\beta \rightarrow \alpha)$

S: $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

$\alpha = x$, $\beta = x \rightarrow x$ e $\gamma = x$ em S:

$$\Big(x \rightarrow \big((x \rightarrow x) \rightarrow x\big)\Big) \rightarrow \Big(\big(x \rightarrow (x \rightarrow x)\big) \rightarrow (x \rightarrow x)\Big)$$

$\alpha = x$ e $\beta = x \rightarrow x$ em K:

$$x \rightarrow \big((x \rightarrow x) \rightarrow x\big)$$

Modus Ponens nas expressões anteriores:

$$\big(x \rightarrow (x \rightarrow x)\big) \rightarrow (x \rightarrow x)$$

$\alpha = x$ e $\beta = x$ em K:

$$x \rightarrow (x \rightarrow x)$$

Modus Ponens nas expressões anteriores:

$$x \rightarrow x$$

Portanto, a identidade é resolvida pela sequência combinatória S K K. Então:

$D = S\ I\ I$

$D = S\ (S\ K\ K)\ (S\ K\ K)$